

LangChain

Interview Preparation Guide

Most Frequently & Commonly Asked Questions
with Detailed Answers and Code Examples

What's inside

30 carefully selected questions spanning fundamentals, LCEL, chains, memory, agents, tools, and Retrieval-Augmented Generation (RAG). Each answer is written to be interview-ready: concise enough to say out loud, deep enough to show real understanding.

Table of Contents

1. Fundamentals & Core Concepts

- Q1. What is LangChain and why is it used?
- Q2. What are the core components / modules of LangChain?
- Q3. What is the difference between an LLM and a Chat Model in LangChain?
- Q4. What are Prompt Templates? Why not just use f-strings?
- Q5. What is LangChain Expression Language (LCEL)?

2. Chains, Runnables & Output Parsing

- Q6. What is the Runnable interface and what are its key methods?
- Q7. Explain RunnablePassthrough, RunnableParallel, and RunnableLambda.
- Q8. What are Output Parsers? Give an example of structured output.
- Q9. What is the difference between Sequential chains and parallel branches?

3. Memory & State

- Q10. Why does an LLM need memory, and how does LangChain provide it?
- Q11. Describe the common types of conversational memory.
- Q12. How do you add message history to an LCEL chain?

4. Retrieval-Augmented Generation (RAG)

- Q13. What is Retrieval-Augmented Generation (RAG) and why is it important?
- Q14. Walk through the end-to-end RAG pipeline in LangChain.
- Q15. What are Document Loaders and Text Splitters?
- Q16. Why is chunk overlap important when splitting text?
- Q17. What are Embeddings and Vector Stores?
- Q18. What is a Retriever, and how does it differ from a Vector Store?
- Q19. How can you reduce hallucinations in a LangChain application?

5. Agents & Tools

- Q20. What are Agents in LangChain?
- Q21. What are Tools, and how do you define one?
- Q22. Explain the ReAct pattern.
- Q23. What is tool calling (function calling) and how is it different from older agents?
- Q24. What is LangGraph and when would you use it over standard agents?

6. Production, Tooling & Comparisons

- Q25. What is LangSmith?
- Q26. How does streaming work in LangChain?
- Q27. How do you handle the LLM context window limit with large documents?

Q28. What is caching in LangChain and why use it?

Q29. How do you implement error handling, retries, and fallbacks?

Q30. When would you choose LangChain vs LlamaIndex vs a direct API call?

1. Fundamentals & Core Concepts

Q1. What is LangChain and why is it used?

LangChain is an open-source framework for building applications powered by Large Language Models (LLMs). It provides a standard, modular interface for connecting LLMs to external data sources, tools, APIs, and memory, so developers do not have to write boilerplate for every model or integration.

It is used because raw LLM API calls are stateless and isolated. Real applications need more: chaining multiple calls together, injecting retrieved documents, remembering conversation history, calling external tools, and parsing structured output. LangChain standardises all of these into composable building blocks.

Key value propositions:

- **Abstraction** — swap between OpenAI, Anthropic, Hugging Face, etc. with minimal code change.
- **Composability** — combine prompts, models, retrievers, and parsers into pipelines.
- **Ecosystem** — hundreds of integrations for vector stores, loaders, and tools.

Q2. What are the core components / modules of LangChain?

LangChain is organised around a few core abstractions that you compose together:

- **Models** — LLMs (text-in/text-out) and Chat Models (message-in/message-out), plus Embedding models.
- **Prompts** — PromptTemplate and ChatPromptTemplate for building dynamic, reusable prompts.
- **Chains** — sequences of calls. Modern LangChain expresses these via LCEL (the pipe operator).
- **Memory** — persists state/conversation history across calls.
- **Retrievers / Indexes** — document loaders, text splitters, embeddings, and vector stores that power RAG.
- **Agents & Tools** — let the LLM decide which actions/tools to invoke.
- **Output Parsers** — convert raw text into structured data (JSON, Pydantic, lists).
- **Callbacks** — hooks for logging, streaming, monitoring (e.g. LangSmith).

Q3. What is the difference between an LLM and a Chat Model in LangChain?

Both are model wrappers, but they differ in their input/output interface:

- **LLM** — a text-completion model. Input is a single string, output is a string. Interface: `llm.invoke("some text")`.
- **Chat Model** — designed for conversational APIs. Input is a list of structured messages (SystemMessage, HumanMessage, AIMessage), and output is an AIMessage.

Most modern models (GPT-4, Claude, Gemini) are chat models, so ChatModel is the common choice. Messages carry roles, which lets you separate system instructions from user input cleanly.

```

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

model = ChatOpenAI(model="gpt-4o-mini")
response = model.invoke([
    SystemMessage(content="You are a helpful tutor."),
    HumanMessage(content="Explain recursion in one line."),
])
print(response.content)

```

Q4. What are Prompt Templates? Why not just use f-strings?

A Prompt Template is a reusable, parameterised recipe for generating prompts. Instead of hardcoding text, you define placeholders that get filled at runtime.

They are preferred over plain f-strings because they are **composable** (they plug directly into chains as Runnables), support **validation** of input variables, allow **partial** formatting, can be saved/loaded and shared via the LangChain Hub, and handle chat message structure (roles) cleanly.

```

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}."),
    ("human", "Answer this: {question}"),
])

msg = prompt.invoke({"domain": "biology", "question": "What is DNA?"})

```

Q5. What is LangChain Expression Language (LCEL)?

LCEL is the declarative way to compose LangChain components using the pipe (|) operator. Every component implements the **Runnable** interface, so the output of one is piped as the input of the next, forming a chain.

LCEL chains get powerful features for free: **streaming**, **async**, **batch** processing, **parallel** execution, retries, fallbacks, and full tracing in LangSmith — all from the same composed object.

```

from langchain_core.output_parsers import StrOutputParser

# prompt -> model -> parser
chain = prompt | model | StrOutputParser()

chain.invoke({"domain": "physics", "question": "What is entropy?"})
chain.batch([{"..."}, {"..."}])      # batch
chain.stream({"..."})                  # streaming

```

2. Chains, Runnables & Output Parsing

Q6. What is the Runnable interface and what are its key methods?

Runnable is the universal protocol that every LCEL component implements. It is what makes components interchangeable and pipeable. The core methods are:

- `invoke()` — run on a single input.
- `batch()` — run on a list of inputs (parallelised).
- `stream()` — yield output chunks as they are produced.
- Each has an async counterpart: `ainvoke`, `abatch`, `astream`.

Because everything shares this interface, a prompt, a model, a retriever, and a parser can all be wired together with the same pipe syntax.

Q7. Explain RunnablePassthrough, RunnableParallel, and RunnableLambda.

These are LCEL helpers for shaping data as it flows through a chain:

- **RunnablePassthrough** — passes input through unchanged; often used to keep the original input alongside computed values (e.g. keeping the user question while also retrieving context).
- **RunnableParallel** (a dict) — runs multiple runnables on the same input concurrently and returns a dict of their outputs.
- **RunnableLambda** — wraps an arbitrary Python function so it becomes a Runnable you can pipe.

```
from langchain_core.runnables import (
    RunnableParallel, RunnablePassthrough, RunnableLambda)

setup = RunnableParallel(
    context=retriever,          # fetch docs
    question=RunnablePassthrough(), # keep raw question
)
chain = setup | prompt | model | StrOutputParser()
```

Q8. What are Output Parsers? Give an example of structured output.

Output parsers convert the raw text an LLM returns into a usable, structured format. LLMs output strings; parsers turn those into lists, JSON, or typed objects, and can also inject format instructions into the prompt.

The most robust approach today is **structured output** via Pydantic, which validates the result against a schema.

```
from pydantic import BaseModel, Field

class Movie(BaseModel):
    title: str = Field(description="film title")
    year: int = Field(description="release year")

# Bind the schema directly to the model
structured_model = model.with_structured_output(Movie)
result = structured_model.invoke("Tell me about Inception")
print(result.title, result.year) # -> typed Movie object
```

Q9. What is the difference between Sequential chains and parallel branches?

A **sequential** flow runs steps one after another, where each step's output feeds the next (prompt → model → parser). This is the default in a piped LCEL chain.

A **parallel** branch (`RunnableParallel`) runs several independent steps on the same input simultaneously and merges their results into a dictionary. Use parallel execution to reduce latency when steps don't depend on each other — for example, summarising a document and extracting keywords from it at the same time.

3. Memory & State

Q10. Why does an LLM need memory, and how does LangChain provide it?

LLM API calls are **stateless** — the model has no recollection of previous turns. To hold a coherent conversation, you must resend the relevant history with each request. Memory is the mechanism that stores and re-injects that history.

In modern LangChain, conversation history is typically managed with **`RunnableWithMessageHistory`** (and `LangGraph` for more complex state), which wraps a chain and automatically loads/saves messages keyed by a session ID.

Q11. Describe the common types of conversational memory.

Historically LangChain offered several memory strategies; understanding the trade-offs is a common interview point:

- **Buffer memory** — stores the full conversation verbatim. Simple but grows unbounded and can blow the context window.
- **Window (buffer-window) memory** — keeps only the last k exchanges. Bounds token usage but forgets older context.
- **Summary memory** — uses an LLM to progressively summarise older turns, keeping a compact running summary. Saves tokens but can lose detail.
- **Summary-buffer memory** — a hybrid: keeps recent turns verbatim and summarises the rest.
- **Vector-store-backed memory** — embeds past messages and retrieves only the most relevant ones, useful for very long histories.

Q12. How do you add message history to an LCEL chain?

Wrap the chain with **`RunnableWithMessageHistory`**, providing a function that returns a history store for a given session ID. The wrapper handles loading prior messages and appending new ones.

```

from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

store = {}
def get_history(session_id):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

chat = RunnableWithMessageHistory(chain, get_history,
                                input_messages_key="question", history_messages_key="history")

chat.invoke({"question": "Hi!"},
            config={"configurable": {"session_id": "user-1"}})

```

4. Retrieval-Augmented Generation (RAG)

Q13. What is Retrieval-Augmented Generation (RAG) and why is it important?

RAG is a technique that supplies an LLM with relevant external information at query time, so its answer is grounded in that data rather than only its training parameters. The retrieved context is inserted into the prompt before generation.

Why it matters:

- **Reduces hallucination** — answers are grounded in real, retrieved documents.
- **Up-to-date knowledge** — access data newer than the model's training cutoff.
- **Private/domain data** — answer questions about internal documents the model never saw.
- **Cost-effective** — cheaper and faster than fine-tuning for knowledge injection, and easy to update by changing the documents.

Q14. Walk through the end-to-end RAG pipeline in LangChain.

A RAG pipeline has two phases. **Indexing** (offline) prepares the knowledge base; **retrieval & generation** (online) answers queries.

Indexing:

- **Load** — Document Loaders pull in source data (PDF, web, DB).
- **Split** — Text Splitters chunk documents into manageable passages.
- **Embed** — an embedding model converts chunks into vectors.
- **Store** — vectors are saved in a Vector Store.

Retrieval & Generation:

- **Retrieve** — the user query is embedded and the most similar chunks are fetched.
- **Augment** — retrieved chunks are inserted into the prompt as context.
- **Generate** — the LLM produces a grounded answer.

```

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
rag_chain.invoke("What is our refund policy?")

```

Q15. What are Document Loaders and Text Splitters?

Document Loaders ingest data from a source and return standardised Document objects (page content + metadata). LangChain has loaders for PDFs, CSVs, web pages, Notion, S3, databases, and many more.

Text Splitters break large documents into smaller chunks so they fit the embedding model and the LLM context window, and so retrieval is more precise. The **RecursiveCharacterTextSplitter** is the common default — it tries to split on paragraph, then line, then word boundaries to keep chunks semantically coherent.

```

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,  # overlap preserves context across chunks
)
chunks = splitter.split_documents(documents)

```

Q16. Why is chunk overlap important when splitting text?

Chunk overlap repeats a small portion of text between consecutive chunks. Without it, a sentence or idea split exactly at a chunk boundary would be torn in half, and neither chunk would contain the complete thought — hurting retrieval quality and the model's ability to answer.

Overlap (commonly 10–20% of chunk size) ensures that context bleeding across a boundary still appears intact in at least one chunk, at the cost of slightly more storage and some duplication.

Q17. What are Embeddings and Vector Stores?

Embeddings are numerical vector representations of text that capture semantic meaning — texts with similar meaning have vectors that are close together in the vector space. They are produced by an embedding model.

Vector Stores are databases optimised to store these vectors and perform fast **similarity search** (typically cosine similarity or dot product). Given a query vector, they return the nearest stored vectors. Popular options include FAISS (local), Chroma, Pinecone, Weaviate, Qdrant, and pgvector.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
docs = vectorstore.similarity_search("query text", k=4)
```

Q18. What is a Retriever, and how does it differ from a Vector Store?

A **Retriever** is a more general interface whose only job is: given a query string, return relevant documents. A vector store is one *source* a retriever can sit on top of (via `.as_retriever()`), but retrievers are broader.

Other retriever types include the **MultiQueryRetriever** (generates several query variations), **Contextual Compression** (filters/condenses retrieved docs), **Ensemble** (combines multiple retrievers), and **Self-Query** (translates natural language into metadata filters). The key distinction: a vector store stores and searches vectors; a retriever is the abstract 'fetch relevant docs' contract.

Q19. How can you reduce hallucinations in a LangChain application?

Several strategies, often combined:

- **Use RAG** — ground answers in retrieved, authoritative documents.
- **Instruct the model** to answer only from the provided context and to say 'I don't know' when the context is insufficient.
- **Cite sources** — return the document chunks used so answers are verifiable.
- **Improve retrieval** — better chunking, re-ranking, and metadata filtering so the right context reaches the model.
- **Lower temperature** — reduces creative drift for factual tasks.
- **Validate output** — use structured output parsers and guardrails to constrain responses.

5. Agents & Tools

Q20. What are Agents in LangChain?

An Agent is a system where the LLM acts as a **reasoning engine** that decides *which* actions to take and in *what* order, rather than following a hardcoded sequence. Given a goal and a set of tools, the agent loops: it reasons about the next step, chooses a tool, observes the result, and repeats until it can produce a final answer.

This dynamic control flow is the key difference from a chain. A chain's steps are fixed at design time; an agent's path is determined at runtime by the model.

Q21. What are Tools, and how do you define one?

A Tool is a function the agent can call to interact with the outside world — a web search, a calculator, a database query, an API, or custom business logic. Each tool has a name, a description (which the LLM reads to decide when to use it), and an input schema.

The simplest way to create one is the `@tool` decorator. A clear docstring is critical because the model relies on it to choose the tool.

```
from langchain_core.tools import tool

@tool
def get_word_length(word: str) -> int:
    """Return the number of characters in a word."""
    return len(word)

# Bind tools to a model for tool-calling
model_with_tools = model.bind_tools([get_word_length])
```

Q22. Explain the ReAct pattern.

ReAct stands for **Reasoning + Acting**. It is an agent prompting pattern where the model interleaves *Thought* (reasoning about what to do), *Action* (choosing a tool and input), and *Observation* (the tool's result) in a loop.

The cycle — Thought → Action → Observation → Thought → ... → Final Answer — lets the model break a complex task into steps, use tools to gather facts, and self-correct based on what it observes, rather than answering in a single shot.

Q23. What is tool calling (function calling) and how is it different from older agents?

Tool calling is a native capability of modern LLMs: the model is given tool schemas and can output a **structured request** to call one or more tools with specific arguments. LangChain exposes this via `model.bind_tools([...])`.

Older text-based agents (e.g. classic ReAct) relied on the model emitting a specific text format that then had to be parsed — brittle and error-prone. Native tool calling is far more reliable because the structure is enforced by the model provider, supports parallel tool calls, and integrates cleanly with structured output.

Q24. What is LangGraph and when would you use it over standard agents?

LangGraph is a LangChain library for building **stateful, multi-step** applications as a graph of nodes and edges. Nodes are functions/LLM calls; edges (including conditional edges) define control flow; and a shared state object is passed between them.

Use LangGraph when you need **cycles**, fine-grained control over an agent's loop, human-in-the-loop checkpoints, persistence, or complex multi-agent orchestration — scenarios where a linear LCEL chain or a black-box agent executor is too limiting. It has become the recommended foundation for

production agents.

6. Production, Tooling & Comparisons

Q25. What is LangSmith?

LangSmith is LangChain's platform for **observability, debugging, testing, and evaluation** of LLM applications. It traces every step of a chain or agent — inputs, outputs, latency, token usage, and tool calls — so you can see exactly what happened and why.

It is widely used because LLM apps are non-deterministic and hard to debug. LangSmith adds dataset-based evaluation, prompt comparison, and monitoring for production.

Q26. How does streaming work in LangChain?

Streaming yields the model's output token-by-token (or chunk-by-chunk) as it is generated, instead of waiting for the full response — greatly improving perceived latency in chat UIs. Because every Runnable supports `.stream()` and `.astream()`, any LCEL chain can stream end-to-end.

```
for chunk in chain.stream({"question": "Explain gravity"}):
    print(chunk, end="", flush=True)

# astream_events() gives granular events from every step
# (useful for streaming intermediate tool calls in agents)
```

Q27. How do you handle the LLM context window limit with large documents?

The context window caps how many tokens fit in one request, so large inputs must be managed. Common strategies:

- **RAG / chunk + retrieve** — only feed the most relevant chunks, not the whole corpus.
- **Map-Reduce** — summarise each chunk independently (map), then combine the summaries (reduce).
- **Refine** — iteratively build an answer, updating it with each new chunk.
- **Summary memory** — compress conversation history to stay within limits.

Q28. What is caching in LangChain and why use it?

Caching stores the results of LLM calls so that identical requests return the saved response instead of hitting the API again. This **cuts cost and latency** for repeated prompts. LangChain supports an in-memory cache for quick experiments and persistent caches (SQLite, Redis, etc.) for production.

```
from langchain_core.globals import set_llm_cache
from langchain_community.cache import InMemoryCache

set_llm_cache(InMemoryCache())
# First call hits the API; identical later calls are served from cache.
```

Q29. How do you implement error handling, retries, and fallbacks?

LCEL provides these as first-class Runnable features, so you don't write manual try/except loops around every call:

- **.with_retry()** — automatically retries a step on transient failures with backoff.
- **.with_fallbacks([...])** — if the primary runnable fails, fall back to alternatives (e.g. a cheaper or different model).

```
robust = (
    primary_model
    .with_retry(stop_after_attempt=3)
    .with_fallbacks([backup_model])
)
```

Q30. When would you choose LangChain vs LlamaIndex vs a direct API call?

A common 'judgement' question. The honest answer compares trade-offs:

- **Direct API call** — best for simple, single-shot prompts. No framework overhead; but you build chaining, memory, and retrieval yourself.
- **LangChain** — best for complex, multi-step applications: agents, tool use, orchestration, and a broad integration ecosystem. More general-purpose.
- **LlamaIndex** — specialises in data indexing and retrieval (RAG). Its strength is sophisticated indexing/query strategies over your data.

They aren't mutually exclusive — LangChain and LlamaIndex are often used together, with LlamaIndex handling retrieval and LangChain orchestrating the wider application.